HUSTLR ARCHITECTURE STANDARDS

Standard Operating Procedures, Component Blueprints & Development Manual

Laravel 13.x | Livewire 4.x | Flux UI 2.x | Tailwind CSS v4 | PHP 8.3+ | Larastan Level 7

Manage Side Hustle (MSH) - Architecture Standards - Version 1.0

1 Executive Architecture & Core Invariants

Hustlr (Manage Side Hustle - MSH) is built upon a high-performance reactive Laravel 13 stack. Every engineer and automated agent must preserve the four core invariants below to prevent architectural drift, performance degradation, and security flaws.

- **UUID Primary Keys:** All domain entities utilize UUID primary keys with HasUuids to ensure distributed scale and obscure sequential database IDs.
- **Strict PHP Typing:** Methods must have explicit parameter and return types. Eloquent relations and paginators must declare PHPStan generics.
- **Livewire 4 + Flux UI:** User-facing dashboard interfaces are written as reactive Livewire 4 components styled with official Flux UI components.
- **Layer Separation:** Controllers are reserved for APIs and webhooks; Livewire handles reactive pages; Services manage domain transactions.

ARCHITECTURAL RULE

Never execute database queries inside Blade views. Eager load all relations in the component or repository using with(['status', 'client', ...]) to completely eliminate N+1 query bottlenecks.

2 Architectural Decision Matrix

Select the correct component layer based on the functional requirement:

Layer	Directory	Primary Responsibility	Key Invariant
Livewire Page	app/Livewire/*	Reactive UI pages, forms, modals	Route::livewire, #[Title], #[Computed]
Controller	app/Http/Controllers/*	Stateless REST APIs, webhooks, files	Form Requests & Eloquent Resources
Service	app/Services/*	Business rules, multi-model workflows	DB::transaction, Dependency Injection
Repository	app/Repositories/*	Complex queries, eager loads, filters	Interface contracts, typed paginators
Action	app/Actions/*	Single discrete domain command (SRP)	Invokable or execute() method
Model	app/Models/*	Domain entities & relationships	HasUuids, $fillable, casts(), generics

3 Domain Models & Database Architecture (UUID Standard)

Domain models must strictly adhere to the UUID primary key format and provide generic PHPDocs to satisfy Larastan Level 7 analysis.

app/Models/Project.php

```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Concerns\HasUuids;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;

class Project extends Model
{
    use HasFactory, HasUuids, SoftDeletes;

    protected $primaryKey = 'project_id';
    public $incrementing = false;
    protected $keyType = 'string';

    protected $fillable = ['project_name', 'description', 'status_id', 'client_id'];

    protected function casts(): array
    {
        return ['start_date' => 'date', 'end_date' => 'date'];
    }

    /** @return BelongsTo<Client, $this> */
    public function client(): BelongsTo
    {
        return $this->belongsTo(Client::class, 'client_id', 'client_id');
    }
}
```

Database Migration Standard

database/migrations/xxxx_xx_xx_create_projects_table.php

```php
Schema::create('projects', function (Blueprint $table) {
    $table->uuid('project_id')->primary();
    $table->string('project_name');
    $table->text('description')->nullable();
    $table->enum('ownership_type', ['Internal', 'Client', 'Freelance'])->default('Inte
    $table->foreignUuid('status_id')->constrained('project_statuses', 'status_id');
    $table->foreignUuid('client_id')->nullable()->constrained('clients', 'client_id')-
    $table->timestamps();
    $table->softDeletes();
    $table->index(['status_id', 'ownership_type']);
});
```

4 Livewire 4 & Flux UI Component Standards

Livewire 4 components provide real-time UI interactivity without writing client-side JavaScript. All component queries must use #[Computed] properties with pagination.

app/Livewire/Projects/Index.php

```php
<?php
namespace App\Livewire\Projects;

use App\Models\Project;
use Flux\Flux;
use Illuminate\Contracts\Pagination\LengthAwarePaginator;
use Livewire\Attributes\Computed;
use Livewire\Attributes\Title;
use Livewire\Attributes\Url;
use Livewire\Component;
use Livewire\WithPagination;

#[Title('Projects Directory')]
class Index extends Component
{
    use WithPagination;

    #[Url] public string $search = '';

    /** @return LengthAwarePaginator<int, Project> */
    #[Computed]
    public function projects(): LengthAwarePaginator
    {
        return Project::query()
            ->with(['status', 'client', 'technologies'])
            ->when($this->search, fn ($q) => $q->where('project_name', 'like', "%{$thi
            ->latest('updated_at')
            ->paginate(10);
    }

    public function delete(string $id): void
    {
        Project::findOrFail($id)->delete();
        Flux::toast(variant: 'success', text: __('Project deleted successfully.'));
    }
}
```

Flux UI Blade Template Standard

resources/views/livewire/projects/index.blade.php

```
<div>
    <div class="flex justify-between items-center mb-6">
        <flux:heading size="xl">Projects</flux:heading>
        <flux:button href="{{ route('projects.create') }}" variant="primary" icon="plu
            New Project
        </flux:button>
    </div>
    <flux:input wire:model.live.debounce.300ms="search" placeholder="Search..." icon="
    <flux:table class="mt-4">
        <flux:table.rows>
            @foreach ($this->projects as $project)
                <flux:table.row :key="$project->project_id">
                    <flux:table.cell>{{ $project->project_name }}</flux:table.cell>
                    <flux:table.cell><flux:badge>{{ $project->status->status_name }}</
                </flux:table.row>
            @endforeach
        </flux:table.rows>
    </flux:table>
</div>
```

5 Service & Repository Layers

Services coordinate complex business logic and manage database transactions. Repositories isolate data access queries behind clean interface contracts.

app/Services/ProjectService.php

```php
<?php
namespace App\Services;

use App\Models\Project;
use App\Repositories\Contracts\ProjectRepositoryInterface;
use Illuminate\Support\Facades\DB;

class ProjectService
{
    public function __construct(protected ProjectRepositoryInterface $projectRepo) {}

    /** @param array<string, mixed> $data */
    public function createProject(array $data, string $creatorId): Project
    {
        return DB::transaction(function () use ($data, $creatorId): Project {
            $data['created_by'] = $creatorId;
            $technologies = $data['technologies'] ?? [];
            unset($data['technologies']);

            $project = $this->projectRepo->create($data);
            if (!empty($technologies)) {
                $project->technologies()->sync($technologies);
            }
            return $project->fresh(['status', 'client', 'technologies']);
        });
    }
}
```

app/Repositories/Eloquent/ProjectRepository.php

```php
<?php
namespace App\Repositories\Eloquent;

use App\Models\Project;
use App\Repositories\Contracts\ProjectRepositoryInterface;
use Illuminate\Contracts\Pagination\LengthAwarePaginator;

class ProjectRepository implements ProjectRepositoryInterface
{
    /** @return LengthAwarePaginator<int, Project> */
    public function getPaginated(array $filters = [], int $perPage = 10): LengthAwareP
    {
        return Project::query()
            ->with(['status', 'client', 'technologies'])
            ->when($filters['search'] ?? null, fn ($q, $s) => $q->where('project_name'
            ->latest('updated_at')->paginate($perPage);
    }
}
```

6 Controllers, Form Requests & RBAC

Controllers handle HTTP requests, Form Requests validate input with permission gates, and Hustlr's PermissionManager orchestrates granular role permissions.

app/Http/Controllers/Api/ProjectController.php

```php
<?php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\StoreProjectRequest;
use App\Http\Resources\ProjectResource;
use App\Services\ProjectService;
use Illuminate\Http\JsonResponse;
use Symfony\Component\HttpFoundation\Response;

class ProjectController extends Controller
{
    public function __construct(protected ProjectService $projectService) {}

    public function store(StoreProjectRequest $request): JsonResponse
    {
        $project = $this->projectService->createProject($request->validated(), $reques
        return (new ProjectResource($project))->response()->setStatusCode(Response::HT
    }
}
```

app/Http/Requests/StoreProjectRequest.php

```php
public function authorize(): bool
{
    return $this->user()?->can('IPS - PROJECT - Can Create Projects') ?? false;
}
public function rules(): array
{
    return [
        'project_name' => ['required', 'string', 'max:255'],
        'status_id'    => ['required', 'uuid', 'exists:project_statuses,status_id'],
        'client_id'    => ['nullable', 'uuid', 'exists:clients,client_id'],
    ];
}
```

PERMISSION MANAGEMENT PATTERN

Add new permissions to app/PermissionManager/IntegratedPersonalSystem/ role classes (Owner, Management, Staff). Always prefix permissions with 'IPS - {MODULE} - Can {Action} {Target}'.

7 Automated Testing Standards (PHPUnit 12)

Hustlr uses PHPUnit 12. Write feature tests for all business logic, authorization rules, and Livewire component interactions.

tests/Feature/ProjectIndexTest.php

```php
<?php
namespace Tests\Feature;

use App\Livewire\Projects\Index;
use App\Models\Project;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Livewire\Livewire;
use Tests\TestCase;

class ProjectIndexTest extends TestCase
{
    use RefreshDatabase;

    public function test_user_can_search_projects_in_livewire(): void
    {
        $user = User::factory()->create();
        Project::factory()->create(['project_name' => 'Project Alpha']);
        Project::factory()->create(['project_name' => 'Project Beta']);

        $this->actingAs($user);

        Livewire::test(Index::class)
            ->set('search', 'Alpha')
            ->assertSee('Project Alpha')
            ->assertDontSee('Project Beta');
    }
}
```

8 Quality Gates & Artisan Cheatsheet

Mandatory pre-commit verification scripts:

- **Code Formatter:** composer run lint (Laravel Pint agent-formatted)
- **Static Analysis:** composer run types:check (Larastan PHPStan Level 7)
- **Full Suite:** composer run test (Cache clear, lint, types, and PHPUnit)

Artisan Scaffolding Commands

Scaffold Target	Artisan CLI Command
Model + Factory + Migration + Seeder	php artisan make:model Task -m -f -s
Livewire 4 Page Component	php artisan make:livewire Tasks/Index
Service Layer Class	php artisan make:class Services/TaskServ
Repository Implementation	php artisan make:class Repositories/Eloq
Discrete Action Command	php artisan make:class Actions/Tasks/Ass
Form Request Validation	php artisan make:request StoreTaskReques
PHPUnit 12 Feature Test	php artisan make:test TaskFeatureTest --